The Spinach Optimal Control Module

User Manual

Talos*

27 August 2026

Spinach 2.12, optimal control module architecture of pull request #332

Contents

*AI research assistant, Weizmann Institute of Science. The module itself is the work of David Goodwin, Uluk Rasulov, Marcel Keitel, Ilya Kuprov, and the Spinach developer team.

1 Physics and mathematics of GRAPE

1.1 The quantum optimal control problem

Magnetic resonance systems evolve under an equation of motion that Spinach always writes in the form

$$\frac{d}{dt}\boldsymbol{\rho}(t) = -i\,\mathbf{L}(t)\,\boldsymbol{\rho}(t), \qquad \boldsymbol{\rho}(t) = \exp_{(\mathrm{o})}\left[-i\int_0^t \mathbf{L}(\tau)\,d\tau\right]\boldsymbol{\rho}(0), \tag{1}$$

where $\exp_{(\mathrm{o})}$ denotes a time-ordered exponential. In Hilbert space $\boldsymbol{\rho}$ is a density matrix and $\mathbf{L}$ is the commutation superoperator with a Hermitian Hamiltonian, so that the propagation over an interval of constant $\mathbf{H}$ is a unitary sandwich

$$\boldsymbol{\rho}(t+\Delta t) = \mathbf{P}\,\boldsymbol{\rho}(t)\,\mathbf{P}^{\dagger}, \qquad \mathbf{P} = \exp(-i\mathbf{H}\Delta t). \tag{2}$$

In Liouville space $\boldsymbol{\rho}$ is a state vector and $\mathbf{L}$ is a Liouvillian that may contain relaxation and chemical kinetics; it is then not Hermitian, the propagation

$$\boldsymbol{\rho}(t+\Delta t) = \exp(-i\mathbf{L}\Delta t)\,\boldsymbol{\rho}(t) \tag{3}$$

is not unitary, and dissipative dynamics is handled without further ceremony. This is the principal reason why the workhorse formalism of the optimal control module is Liouville space.

The control problem is stated as follows. The evolution generator is split into an uncontrollable *drift* and a linear combination of *control operators* $\{\mathbf{C}_k\}$, $k = 1, \ldots, K$, whose coefficients are at the disposal of the instrument:

$$\mathbf{L}(t) = \mathbf{L}_0(t) + \sum_{k=1}^{K} c_k(t)\,\mathbf{C}_k. \tag{4}$$

Typical control operators are $\hat{L}_X$ and $\hat{L}_Y$ magnetisation operators on the channels of the instrument; the drift contains chemical shifts, couplings, exchange, and relaxation. The pulse of total duration T is discretised over N intervals $\Delta t_1, \ldots, \Delta t_N$. Two discretisations of $c_k(t)$ are supported:

Piecewise-constant (`rectangle`). The coefficient $c_k(t) = c_{kn}$ is constant on the n-th interval; the waveform array has N columns. This is the classical GRAPE setting of Khaneja *et al.* [1].

Piecewise-linear (`trapezium`). The coefficient is a linear interpolation between *nodal* values c_{kn} specified at the $N{+}1$ interval edges; the waveform array has $N{+}1$ columns. This matches what waveform generators actually emit, and is integrated with a product quadrature (Section 1.6) [7].

In Spinach the user-facing waveform is dimensionless and order unity: the physical coefficient is $c_{kn} = p\,u_{kn}$, where p is the nominal control power (rad/s) supplied in `control.pwr_levels`, and $u_{kn} \in [-1, 1]$ is the normalised waveform that the optimiser manipulates. All derivatives reported to the optimiser are with respect to u_{kn}; conversions are handled internally by the chain rule.

Starting from an initial state $\boldsymbol{\rho}_0$, the pulse should arrive as closely as possible at a target $\boldsymbol{\sigma}$. With the overlap

$$o = \langle\boldsymbol{\sigma}|\boldsymbol{\rho}(T)\rangle = \begin{cases} \boldsymbol{\sigma}^{\dagger}\boldsymbol{\rho}(T) & \text{(Liouville space)} \\ \mathrm{Tr}\left[\boldsymbol{\sigma}^{\dagger}\boldsymbol{\rho}(T)\right] & \text{(Hilbert space)} \end{cases} \tag{5}$$

three fidelity functionals are supported (`control.fidelity`):

$$f_{\mathrm{real}} = \mathrm{Re}\,o, \qquad f_{\mathrm{imag}} = \mathrm{Im}\,o, \qquad f_{\mathrm{square}} = |o|^2. \tag{6}$$

When states are normalised, $f_{\text{real}} \in [-1, +1]$ and $f_{\text{square}} \in [0, 1]$; the square functional is insensitive to the overall phase of the final state. The optimiser maximises the total objective

$$F(\mathbf{u}) = f(\mathbf{u}) - \sum_j w_j \, P_j(\mathbf{u}), \tag{7}$$

where P_j are penalty functionals (Section 1.9) with non-negative weights w_j that hold the waveform inside instrumental constraints. In ensemble optimisations (Section 1.8) the fidelity is additionally averaged over a set of system instances.

GRAPE — gradient ascent pulse engineering [1] — is the observation that, for piecewise-defined waveforms, the gradient $\partial f / \partial u_{kn}$ (and, with more effort, the Hessian) can be computed *analytically* at a cost comparable to that of the simulation itself, after which any standard numerical maximiser converges rapidly. Spinach implements first-order (LBFGS) and second-order (Newton, with two Hessian assembly strategies) optimisers over both integrators and both spaces; the derivations follow.

1.2 Directional derivatives of the matrix exponential

Everything in GRAPE reduces to derivatives of interval propagators $\mathbf{P} = \exp[-i(\mathbf{A} + \varepsilon\mathbf{B})\Delta t]$ with respect to ε at $\varepsilon = 0$ — *directional* derivatives of the matrix exponential in the direction $\mathbf{B}$. Differentiating the exponential series term by term,

$$\mathcal{D}_{\mathbf{B}} \exp(-i\mathbf{A}\Delta t) = \frac{\partial}{\partial \varepsilon} e^{-i(\mathbf{A}+\varepsilon\mathbf{B})\Delta t}\bigg|_{\varepsilon=0} = -i \int_0^{\Delta t} e^{-i\mathbf{A}(\Delta t - \tau)} \, \mathbf{B} \, e^{-i\mathbf{A}\tau} \, d\tau, \tag{8}$$

a nested integral that would be expensive head-on. The auxiliary matrix technique of Van Loan [6] and Najfeld and Havel [5], in the form of Eq. 16 of Goodwin and Kuprov [3], obtains it from a single exponential of a block-bidiagonal matrix:

$$\exp\left(-i \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{0} & \mathbf{A} \end{bmatrix} \Delta t\right) = \begin{bmatrix} \mathbf{P} & \mathcal{D}_{\mathbf{B}}\mathbf{P} \\ \mathbf{0} & \mathbf{P} \end{bmatrix}, \qquad \mathbf{P} = e^{-i\mathbf{A}\Delta t}. \tag{9}$$

The block structure telescopes: with N diagonal blocks $\mathbf{A}$ and directions $\mathbf{B}_1, \ldots, \mathbf{B}_{N-1}$ on the superdiagonal, the top-right corner of the exponential contains the nested integral of the ordered product of all the directions; for equal directions $\mathbf{B}$ it equals the n-th derivative divided by $n!$. In particular, the 3×3 block matrix

$$\exp\left(-i \begin{bmatrix} \mathbf{A} & \mathbf{B}_1 & \mathbf{0} \\ \mathbf{0} & \mathbf{A} & \mathbf{B}_2 \\ \mathbf{0} & \mathbf{0} & \mathbf{A} \end{bmatrix} \Delta t\right) \tag{10}$$

delivers in its top-right block the mixed second directional derivative (the ordered half; the full mixed derivative is the sum of both orderings, and for $\mathbf{B}_1{=}\mathbf{B}_2{=}\mathbf{B}$ the block is $\frac{1}{2}\mathcal{D}^2_{\mathbf{B}}\mathbf{P}$). The kernel utility `dirdiff.m` implements exactly this: it builds the N-block auxiliary matrix, exponentiates it with the standard Spinach `propagator()` machinery at tightened tolerance, and returns $\{\mathbf{P}, \mathcal{D}\mathbf{P}, \mathcal{D}^2\mathbf{P}, \ldots\}$ with the factorial factors restored.

Two properties make this exact and cheap: the auxiliary matrix is highly sparse when $\mathbf{A}$ and $\mathbf{B}$ are sparse, and — crucially for Liouville space — the derivative is never needed as a matrix, only its *action on a state*. Acting with Eq. (9) on a stacked vector,

$$\exp\left(-i \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{0} & \mathbf{A} \end{bmatrix} \Delta t\right) \begin{bmatrix} \mathbf{0} \\ \boldsymbol{\rho} \end{bmatrix} = \begin{bmatrix} (\mathcal{D}_{\mathbf{B}}\mathbf{P})\,\boldsymbol{\rho} \\ \mathbf{P}\,\boldsymbol{\rho} \end{bmatrix}, \tag{11}$$

so one call to the Spinach matrix-exponential-times-vector routine `step()` on a doubled-dimension sparse generator returns the derivative applied to the current state. This *auxiliary vector trick* is the computational core of the Liouville-space GRAPE gradient.

1.3 GRAPE in Liouville space

Let the piecewise-constant generator on interval n be

$$\mathbf{L}_n = \mathbf{L}_0^{(n)} + \sum_{k=1}^{K} c_{kn}\mathbf{C}_k, \qquad \mathbf{P}_n = \exp(-i\mathbf{L}_n\Delta t_n), \tag{12}$$

where the drift $\mathbf{L}_0^{(n)}$ cycles through the user-supplied drift array (one drift, or one per time slice for time-dependent drifts, e.g. under magic angle spinning). The final state is $\boldsymbol{\rho}(T) = \mathbf{P}_N \cdots \mathbf{P}_1\boldsymbol{\rho}_0$ and the overlap is $o = \boldsymbol{\sigma}^\dagger\mathbf{P}_N \cdots \mathbf{P}_1\boldsymbol{\rho}_0$. Differentiating with respect to c_{kn} touches only $\mathbf{P}_n$:

$$\frac{\partial o}{\partial c_{kn}} = \boldsymbol{\sigma}^\dagger\,\mathbf{P}_N \cdots \mathbf{P}_{n+1}\,(\mathcal{D}_{\mathbf{C}_k}\mathbf{P}_n)\,\mathbf{P}_{n-1}\cdots\mathbf{P}_1\boldsymbol{\rho}_0 = \boldsymbol{\chi}_n^\dagger\,(\mathcal{D}_{\mathbf{C}_k}\mathbf{P}_n)\,\boldsymbol{\rho}_{n-1}, \tag{13}$$

where the *forward trajectory* and the *adjoint (backward) trajectory* are

$$\boldsymbol{\rho}_n = \mathbf{P}_n \cdots \mathbf{P}_1\boldsymbol{\rho}_0, \qquad \boldsymbol{\chi}_n = \mathbf{P}_{n+1}^\dagger \cdots \mathbf{P}_N^\dagger\,\boldsymbol{\sigma}. \tag{14}$$

Both trajectories cost one propagation sweep each; all KN gradient elements then follow from Eq. (11) at one doubled-dimension `step()` call each, with the stacked vector $[\mathbf{0}; \boldsymbol{\rho}_{n-1}]$ and the auxiliary generator built from $\mathbf{L}_n$ and $\mathbf{C}_k$. No propagator and no derivative is ever stored as a matrix.

Because Liouvillians need not be Hermitian, $\mathbf{P}_n^\dagger \neq \mathbf{P}_n^{-1}$ in general, and the adjoint trajectory is generated by propagating with the *adjoint generator over negative time*: $\mathbf{P}_n^\dagger = \exp(-i\mathbf{L}_n^\dagger(-\Delta t_n))$. The kernel therefore builds backward generators from adjoint drifts and adjoint control operators and steps them with $-\Delta t$; for Hermitian controls this is invisible, and for deliberately non-Hermitian (dissipative) controls it produces the correct adjoint. The same convention gives correct gradients in the presence of relaxation, where fidelity is genuinely lost along the way.

Keyholes — user-supplied linear idempotent maps (projectors) applied to the state between intervals — commute with this scheme: the forward pass applies the keyhole after each step, the backward pass applies it before the adjoint step at the mirrored position, and since projectors are Hermitian the gradient formula (13) is unchanged. Frozen waveform elements (a logical mask in `control.freeze`) are simply skipped, and their gradient entries set to zero.

1.4 GRAPE in Hilbert space

In the `zeeman-hilb` formalism the state is a density matrix, the propagation is the unitary sandwich of Eq. (2), and all generators are required to be Hermitian (this is enforced at problem setup). The trajectories are

$$\boldsymbol{\rho}_n = \mathbf{P}_n\boldsymbol{\rho}_{n-1}\mathbf{P}_n^\dagger, \qquad \boldsymbol{\chi}_n = \mathbf{P}_{n+1}^\dagger\boldsymbol{\chi}_{n+1}\mathbf{P}_{n+1}, \qquad \boldsymbol{\chi}_N = \boldsymbol{\sigma}, \tag{15}$$

the overlap is the Frobenius inner product $o = \mathrm{Tr}[\boldsymbol{\sigma}^\dagger\boldsymbol{\rho}_N]$, and the product rule applied to the sandwich gives

$$\frac{\partial o}{\partial c_{kn}} = \left\langle \boldsymbol{\chi}_n \;\middle|\; (\mathcal{D}_{\mathbf{C}_k}\mathbf{P}_n)\,\boldsymbol{\rho}_{n-1}\mathbf{P}_n^\dagger + \mathbf{P}_n\boldsymbol{\rho}_{n-1}\,(\mathcal{D}_{\mathbf{C}_k}\mathbf{P}_n)^\dagger \right\rangle, \tag{16}$$

with $\langle\mathbf{A}|\mathbf{B}\rangle = \mathrm{Tr}[\mathbf{A}^\dagger\mathbf{B}]$. Here matrix dimensions are those of Hilbert, not Liouville, space, so propagators and their derivatives are computed and stored as matrices: `dirdiff.m` with two blocks returns $\{\mathbf{P}_n, \mathcal{D}\mathbf{P}_n\}$ in one auxiliary exponential per (k, n) pair. Second derivatives for the Newton optimiser use the three-block auxiliary matrix for $\mathcal{D}^2\mathbf{P}$ (equal and mixed directions) and the second-order product rule for the sandwich,

$$\partial^2(\mathbf{P}\boldsymbol{\rho}\mathbf{P}^\dagger) = (\mathcal{D}^2\mathbf{P})\boldsymbol{\rho}\mathbf{P}^\dagger + 2\,(\mathcal{D}\mathbf{P})\boldsymbol{\rho}(\mathcal{D}\mathbf{P})^\dagger + \mathbf{P}\boldsymbol{\rho}(\mathcal{D}^2\mathbf{P})^\dagger, \tag{17}$$

distributed over interval pairs in the same way as in Liouville space (Section 1.7).

1.5 Fidelity functionals and their derivatives

All propagator differentiation above concerns the complex overlap o and its derivatives $g_{kn} = \partial o/\partial c_{kn}$ and $h_{kn,jm} = \partial^2 o/\partial c_{kn}\partial c_{jm}$. The requested fidelity functional is applied at the very end:

$$f = \mathrm{Re}\, o: \quad \nabla f = \mathrm{Re}\, \mathbf{g}, \quad \nabla^2 f = \mathrm{Re}\, \mathbf{h}; \tag{18}$$

$$f = \mathrm{Im}\, o: \quad \nabla f = \mathrm{Im}\, \mathbf{g}, \quad \nabla^2 f = \mathrm{Im}\, \mathbf{h}; \tag{19}$$

$$f = |o|^2 = o\bar{o}: \quad \nabla f = \bar{o}\,\mathbf{g} + o\,\bar{\mathbf{g}} = 2\,\mathrm{Re}(\bar{o}\,\mathbf{g}), \tag{20}$$

$$\nabla^2 f = \bar{o}\,\mathbf{h} + o\,\bar{\mathbf{h}} + \mathbf{g}\bar{\mathbf{g}}^{\mathrm{T}} + \bar{\mathbf{g}}\mathbf{g}^{\mathrm{T}}. \tag{21}$$

The `real` functional is the default and is appropriate for state-to-state transfer with a fixed target phase; `square` removes the phase sensitivity; `imag` is occasionally useful for quadrature targets.

1.6 The piecewise-linear (trapezium) integrator

Real instruments emit waveforms that ramp linearly between the points of the digital-to-analogue converter. On interval n the generator interpolates linearly between its edge values, $\mathbf{G}(t) = \mathbf{G}_L + (\mathbf{G}_R - \mathbf{G}_L)\, t/\Delta t$, where $\mathbf{G}_L$ and $\mathbf{G}_R$ are built from the nodal control coefficients $c_{k,n}$ and $c_{k,n+1}$. The exact propagator is the time-ordered exponential; expanding it in the Magnus series and keeping the terms through second order gives

$$\begin{aligned}\boldsymbol{\Omega} &= -i\int_0^{\Delta t} \mathbf{G}(t)\, dt + \frac{1}{2}\int_0^{\Delta t}\int_0^{t_1} [-i\mathbf{G}(t_1), -i\mathbf{G}(t_2)]\, dt_2\, dt_1 + \cdots \\ &= -i\Delta t\, \frac{\mathbf{G}_L + \mathbf{G}_R}{2} + \frac{\Delta t^2}{12}\,[\mathbf{G}_L, \mathbf{G}_R] + O(\Delta t^5),\end{aligned} \tag{22}$$

where the double integral was evaluated using $[\mathbf{G}(t_1), \mathbf{G}(t_2)] = \frac{t_2 - t_1}{\Delta t}[\mathbf{G}_L, \mathbf{G}_R]$. The interval propagator is therefore written as a standard exponential of an *effective generator*,

$$\mathbf{P} = \exp(-i\,\mathbf{G}_{\mathrm{eff}}\,\Delta t), \qquad \mathbf{G}_{\mathrm{eff}} = \frac{\mathbf{G}_L + \mathbf{G}_R}{2} + \frac{i\Delta t}{12}\,[\mathbf{G}_L, \mathbf{G}_R]\,, \tag{23}$$

the Iserles–Nørsett product quadrature [8] used throughout Spinach for piecewise-linear propagation [7]. The kernel routine `step()` applies this quadrature when it is given the edge and midpoint generators of an interval; the optimal control code supplies $\{\mathbf{G}_L, (\mathbf{G}_L{+}\mathbf{G}_R)/2, \mathbf{G}_R\}$ for each forward step and the adjoint triple in reverse order for each backward step.

Derivatives require one extra chain-rule stage, because each nodal coefficient now enters the effective generator both linearly and through the commutator. Differentiating Eq. (23) with respect to the left-edge and right-edge coefficients of control k:

$$\frac{\partial \mathbf{G}_{\mathrm{eff}}}{\partial c_k^{(L)}} = \frac{\mathbf{C}_k}{2} + \frac{i\Delta t}{12}\,[\mathbf{C}_k, \mathbf{G}_R] = \frac{\mathbf{C}_k}{2} + \frac{i\Delta t}{12}\left([\mathbf{C}_k, \mathbf{D}_R] + \sum_m c_m^{(R)}\,[\mathbf{C}_k, \mathbf{C}_m]\right), \tag{24}$$

$$\frac{\partial \mathbf{G}_{\mathrm{eff}}}{\partial c_k^{(R)}} = \frac{\mathbf{C}_k}{2} + \frac{i\Delta t}{12}\,[\mathbf{G}_L, \mathbf{C}_k] = \frac{\mathbf{C}_k}{2} + \frac{i\Delta t}{12}\left([\mathbf{D}_L, \mathbf{C}_k] + \sum_m c_m^{(L)}\,[\mathbf{C}_m, \mathbf{C}_k]\right), \tag{25}$$

where $\mathbf{D}_{L,R}$ are the edge drifts. The control–control commutators $[\mathbf{C}_k, \mathbf{C}_m]$ do not depend on the waveform; they are computed once at problem setup, stored in `control.cc_comm`, and a logical table of exactly commuting pairs (`control.cc_comm_idx`) skips the vanishing terms. The kernel function `aux_mat.m` assembles Eqs. (24)–(25) and packs them into the auxiliary block matrices of Section 1.2 with $\mathbf{A} = \mathbf{G}_{\mathrm{eff}}$ and $\mathbf{B} = \partial \mathbf{G}_{\mathrm{eff}}/\partial c$; the propagator derivative then follows from the same auxiliary vector trick, applied per interval edge.

Since every interior node is the right edge of one interval and the left edge of the next, its gradient is a two-term product rule,

$$\frac{\partial o}{\partial c_{kn}} = \boldsymbol{\chi}_n^{\dagger}\left(\frac{\partial \mathbf{P}_n}{\partial c_{kn}}\right)\boldsymbol{\rho}_{n-1} + \boldsymbol{\chi}_{n-1}^{\dagger}\left(\frac{\partial \mathbf{P}_{n-1}}{\partial c_{kn}}\right)\boldsymbol{\rho}_{n-2}, \tag{26}$$

with the first and the last node contributing one term only. The trapezium integrator is available in both spaces; analytic Hessians are not implemented for it, so the optimiser choices are the quasi-Newton family (Section 2.2).

1.7 Analytic Hessians

For the piecewise-constant integrator, second derivatives of the overlap come in two kinds. *Same-interval* elements need the second directional derivative of a single propagator,

$$\frac{\partial^2 o}{\partial c_{kn}\partial c_{jn}} = \boldsymbol{\chi}_n^{\dagger}\,\mathcal{D}^2_{\mathbf{C}_k\mathbf{C}_j}\mathbf{P}_n\,\boldsymbol{\rho}_{n-1}, \tag{27}$$

obtained from the three-block auxiliary matrix of Section 1.2; the code doubles the extracted ordered block, which for $k = j$ is exactly $\mathcal{D}^2\mathbf{P}$ and for $k \neq j$ combines with the transposed assembly below into the symmetric mixed derivative. *Cross-interval* elements ($m < n$) factor into two first derivatives separated by ordinary propagation,

$$\frac{\partial^2 o}{\partial c_{kn}\partial c_{jm}} = \boldsymbol{\chi}_n^{\dagger}\,(\mathcal{D}_{\mathbf{C}_k}\mathbf{P}_n)\,\mathbf{P}_{n-1}\cdots\mathbf{P}_{m+1}\,(\mathcal{D}_{\mathbf{C}_j}\mathbf{P}_m)\,\boldsymbol{\rho}_{m-1}. \tag{28}$$

Two assembly strategies are implemented:

`newton`. The derivative actions $(\mathcal{D}_{\mathbf{C}_j}\mathbf{P}_m)\boldsymbol{\rho}_{m-1}$ are stored for all (j, m) during a single sweep; then, for each n, the left factor $\boldsymbol{\chi}_n^{\dagger}(\mathcal{D}_{\mathbf{C}_k}\mathbf{P}_n)$ is propagated *backwards* interval by interval with the adjoint generators, forming inner products with the stored right factors along the way. The cost is one backward sweep per (k, n) pair, all of it matrix-vector.

`goodwin`. Goodwin's acceleration [4] projects every derivative action to the common time origin using cumulative propagators $\mathbf{Q}_n = \mathbf{P}_n\cdots\mathbf{P}_1$: Eq. (28) becomes a single inner product between $\mathbf{Q}_{n-1}$-projected left rows and $\mathbf{Q}$-projected right columns, at the price of computing and storing the cumulative propagators as matrices. It trades memory for speed on problems where the propagators are affordable; it also requires unitary propagation, so all generators must be Hermitian (checked at setup).

The assembled Hessian is symmetrised, the rows and columns of frozen waveform elements are cleared with unit diagonal, and the fidelity chain rule of Section 1.5 is applied. Analytic Hessians are available in both spaces for the rectangle integrator in ordinary (non-steady-state, undistorted) optimisations.

1.8 Ensemble optimal control

A pulse is useful when it works over the instrument and sample inhomogeneities. Spinach optimises the *ensemble average fidelity*

$$\bar{f} = \frac{1}{M}\sum_{i=1}^{M} f_i, \qquad \nabla\bar{f} = \frac{1}{M}\sum_{i=1}^{M}\nabla f_i, \tag{29}$$

where each ensemble member i is one fully specified simulation — a *case*. Cases are enumerated as the Cartesian product of six ensemble dimensions:

dimension	source
initial–target state pair	`control.rho_init` / `rho_targ` cells
drift generator	`control.drifts` cell array
control power level	`control.pwr_levels` vector (B_1 miscalibration)
resonance offset combination	`control.offsets` / `off_ops` (B_0 distribution)
phase cycle line	`control.phase_cycle` rows
distortion function set	`control.distortion` rows

Offsets enter by adding $2\pi\,\delta_\nu\,\mathbf{O}_c$ to the drift of the case, where δ_ν is the offset value in Hz on channel c and $\mathbf{O}_c$ the corresponding offset operator (multiple channels form their own Cartesian product). Power levels scale the physical waveform, $c = p_i u$, so by the chain rule the reported gradient acquires a factor p_i and the Hessian p_i^2. Phase cycling multiplies the initial state, the target state, and each X/Y control pair by case-specific phases (Section 1.11), with the inverse phases applied to the returned gradient and Hessian. Distortion chains and their Jacobians are described in Section 2.5.

Three *ensemble correlations* (`control.ens_corrs`) restrict the Cartesian product to physically correlated combinations: `rho_ens` gives each surviving case its own state pair, `rho_drift` pairs state i with drift i, and `power_drift` pairs power i with drift i. An *ensemble budget* (`control.budget`) draws a reproducible random subset of the full case list — either a member count or a fraction — for stochastic robustness at fixed cost. The bookkeeping lives in a six-column integer *catalog* built once at problem setup by `ens_catalog.m` (Section 3); the ensemble average runs over the catalog rows, and summation order is fixed, so results are deterministic and independent of the parallel pool size.

1.9 Penalty functionals

Instrumental limits are enforced softly, by subtracting weighted penalty terms from the fidelity, Eq. (7). With the normalised waveform u_{kn} (K channels, N_t columns), the implemented functionals are:

$$P_{\mathrm{NS}} = \frac{1}{N_t}\sum_{k,n} u_{kn}^2 \qquad \text{norm square: favours low power;} \tag{30}$$

$$P_{\mathrm{DNS}} = \frac{1}{N_t}\sum_{k,n} \left[(\mathbf{u}\mathbf{D}^{\mathrm{T}})_{kn}\right]^2 \qquad \text{derivative norm square: favours smoothness,} \tag{31}$$

where $\mathbf{D}$ is a five-point second-derivative matrix with wall boundary conditions;

$$P_{\mathrm{SNS}} = \frac{1}{N_t}\sum_{k,n} \Big[(u_{kn} - b_{kn}^{\mathrm{ceil}})^2\,\theta(u_{kn} > b_{kn}^{\mathrm{ceil}}) + (u_{kn} - b_{kn}^{\mathrm{floor}})^2\,\theta(u_{kn} < b_{kn}^{\mathrm{floor}})\Big] \tag{32}$$

is the *spillout norm square*: zero inside the bounds $[\,b^{\mathrm{floor}}, b^{\mathrm{ceil}}\,]$ (scalars or per-element arrays, in fractions of the power level) and quadratic outside — the default penalty, with weight 100, keeping the waveform inside the amplifier range without biasing it inside. P_{SNSA} is SNS applied to the *amplitude* $\sqrt{u_X^2 + u_Y^2}$ of each X/Y channel pair after a polar transformation (with the exact polar chain rules for gradient and Hessian): it caps the pulse amplitude while leaving the phase free, which is the physically right constraint for amplitude-limited, phase-agile hardware. All penalties return analytic gradients and Hessians, which the optimiser combines with the fidelity derivatives; penalties act on the ideal normalised waveform, before power scaling and before any distortion chain.

1.10 Stroboscopic steady state

For experiments that repeat a pulse–and–delay block indefinitely (e.g. steady-state DNP), the object of optimisation is not a single transfer but the *stroboscopic steady state* of the period map. With the period propagator $\mathbf{P}_{\mathrm{tot}} = \mathbf{P}_N \cdots \mathbf{P}_1$ (keyholes excluded — the map must be physical), the steady state $\boldsymbol{\rho}_{\mathrm{ss}}$ satisfies $\mathbf{P}_{\mathrm{tot}}\boldsymbol{\rho}_{\mathrm{ss}} = \boldsymbol{\rho}_{\mathrm{ss}}$ and is found by the kernel `steady()` routine; relaxation must be present (in Liouville space with a thermal fixed point) for the map to be contractive. The fidelity is the overlap of a zero-trace observable $\boldsymbol{\sigma}$ with the steady state at the stroboscopic grid, and its gradient requires the derivative of the fixed point. Writing the steady state as the

geometric series of period maps acting on the unit-trace component shows that the correct adjoint seed is the *dressed target*

$$\boldsymbol{\chi}_0: \quad (\mathbf{1}-\mathbf{P}_{\mathrm{tot}})^{\dagger}\,\boldsymbol{\chi}_0 = \boldsymbol{\sigma} \quad \text{on the traceless subspace,} \tag{33}$$

solved with the unit-state singularity excluded (the code works on the subspace orthogonal to the identity; the target must therefore be traceless, which is checked). With $\boldsymbol{\rho}_0$ overwritten by $\boldsymbol{\rho}_{\mathrm{ss}}$ and $\boldsymbol{\sigma}$ by $\boldsymbol{\chi}_0$, the ordinary GRAPE gradient machinery of Section 1.3 then delivers the steady-state fidelity derivative. Steady-state mode requires the `sphten-liouv` formalism, the rectangle integrator, first-order optimisers, and no prefix/suffix/dead-time decorations; the initial state supplied by the user is ignored (with a warning), because the steady state does not depend on it.

1.11 Phase cycles and cooperative pulses

Phase cycling is built into the ensemble machinery: each line of `control.phase_cycle` is $[\varphi_{\mathrm{init}},\ \varphi_1,\ldots,\varphi_{K/2},\ \varphi_{\mathrm{targ}}]$, and the corresponding ensemble case runs with $\boldsymbol{\rho}_0 \to e^{i\varphi_{\mathrm{init}}}\boldsymbol{\rho}_0$, $\boldsymbol{\sigma} \to e^{i\varphi_{\mathrm{targ}}}\boldsymbol{\sigma}$, and each X/Y control pair rotated through its phase, $(u_X + iu_Y) \to e^{i\varphi_k}(u_X + iu_Y)$. The returned gradient (and Hessian) is rotated back through $e^{-i\varphi_k}$, which is the exact chain rule of the rotation. Maximising the phase-cycled average fidelity produces pulses whose *coherence-pathway-selected sum* behaves as specified.

Cooperative pulses (`grape_coop`) push this one step further for phase-modulated point-to-point transfers: two pulses are optimised simultaneously so that each maximises its own fidelity while their *impurities* — the projections of the final states off the target — cancel in the sum of the two experiments. The functional is

$$F = \frac{f_a + f_b}{2} - \left\langle \|(\mathbf{1}-\mathbf{P}_{\sigma})(\boldsymbol{\rho}_a(T) + \boldsymbol{\rho}_b(T))\|^2 \right\rangle, \tag{34}$$

with $\mathbf{P}_{\sigma}$ the projector onto the target state and the average running over the ensemble; the impurity gradient is assembled by a second pass through the phase-GRAPE machinery with the summed impurity as the target. The two phase profiles travel side by side in one optimisation vector.

2 Numerical implementation in Spinach

2.1 Propagation primitives

All optimal control code is built on the two kernel propagation primitives. Given a generator and an interval, `propagator()` returns the matrix $\exp(-i\mathbf{L}\Delta t)$ using scaled-and-squared Taylor exponentiation with aggressive sparsity bookkeeping; small elements are chopped at `tols.prop_chop`, and computed propagators are cached (in memory and, for large objects, on disk) so that repeated requests are cheap. `step()` computes the *action* $\exp(-i\mathbf{L}\Delta t)\,\boldsymbol{\rho}$ on a state without ever forming the propagator, which is the only practical option at Liouville-space dimensions; given a cell array of edge and midpoint generators it applies the piecewise-linear product quadrature of Section 1.6. The optimal control module uses `propagator()` where matrices are genuinely needed (Hilbert space, steady-state period maps, Goodwin cumulative propagators) and `step()` everywhere else.

The auxiliary matrices of Section 1.2 are assembled as sparse blocks — the doubled or tripled dimension is harmless because the top-right coupling block is as sparse as the control operator — and either exponentiated with `propagator()` (Hilbert space, via `dirdiff.m`) or applied to stacked vectors with `step()` (Liouville space). Propagation inside the derivative loops runs with reporting hushed; the outer functions restore the console before printing.

Numerical hygiene points that matter in practice:

- Control operators, offset operators, and drifts are cleaned up (`clean_up`) at problem setup, so numerical dust in the inputs does not degrade the sparsity of the auxiliary matrices.
- Where unitary propagation is mathematically required — Hilbert space, wavefunction formalism, and the Goodwin Hessian — all generators are checked for Hermiticity at setup and refused otherwise.
- Exactly zero fidelity or an exactly zero gradient aborts the run with an explanatory error: the target is unreachable from the source (a symmetry is in the way), or the initial guess is pathologically poor. This check has saved a great many silently useless optimisations.
- The first LBFGS iteration takes a deliberately conservative step, $0.01\,\mathbf{g}/\max|\mathbf{g}|$, because the fidelity landscape near a random guess is often violently nonlinear.

2.2 The optimiser stack

The maximiser `fmaxnewton(spin_system,cost_function,guess)` runs the iteration loop for all methods; the objective is called through `objeval()`, which assembles the total functional of Eq. (7) from the fidelity-and-penalties vector returned by the cost function, keeps evaluation counters, and reshapes between waveform arrays and optimisation vectors.

`lbfgs` (default): limited-memory BFGS ascent [9]. The inverse Hessian is applied implicitly from a history of waveform and gradient differences; the history depth is `control.n_grads` (default 50). No regularisation is possible or needed; memory cost is linear in problem size.

`rbfgs` : full-matrix BFGS pseudo-Hessian, regularised before inversion (below). Useful when the pseudo-Hessian conditioning, not the memory, is the limiting factor.

`newton` : analytic Hessian of Section 1.7, symmetrised and regularised. Quadratic local convergence; each iteration costs one Hessian evaluation.

`goodwin` : the same Newton step with the cross-interval Hessian assembled through cumulative propagators [4]; requires Hermitian generators.

Newton-family Hessians are made safely negative-definite (this is a maximisation) by *rational function optimisation* regularisation [10] in the kernel routine `hessreg()`: the Hessian is embedded in an augmented matrix whose scaled eigenvalue problem shifts the spectrum away from zero, with the scaling iterated (up to `reg_max_iter`, condition number capped at `reg_max_cond`) until the step is well-posed. If a computed direction still fails the ascent test $\mathbf{g}^{\mathrm{T}}\mathbf{d} > 0$, the optimiser falls back to projected steepest ascent for that iteration.

The step length is found by a Fletcher-style two-stage line search [11]: `bracketing()` expands an interval that is guaranteed to contain acceptable points, and `sectioning()` contracts it with cubic interpolation until the strong Wolfe conditions hold — sufficient increase with $c_1 = 10^{-2}$ and curvature with $c_2 = 0.9$ (bracket expansion factor 3, section contraction bounds 0.1 and 0.5; these live in `control.ls_*` and rarely need attention). Termination is on step norm ($\|\alpha\mathbf{d}\|_1 <$ `tol_x`, default 10^{-3}), gradient norm ($\|\mathbf{g}\|_2 <$ `tol_g`, default 10^{-6}), iteration count (`max_iter`, default 100), or a failed line search. Every iteration prints fidelity, penalties, total, step length, gradient norm, and the angle between the search direction and the gradient (SDGA) — a useful health indicator: it should be well below 90° and typically shrinks as curvature information accumulates.

Optional machinery hanging off the loop: a checkpoint file (`control.checkpoint`) saves the current waveform in the scratch directory after every accepted step and restarts from it if the run is interrupted; a video writer (`control.video_file`) grabs the diagnostic figure every iteration; a freeze mask (`control.freeze`) excludes selected waveform elements from the optimisation everywhere — gradient, Hessian, history, and line search.

2.3 Coordinate systems and wrappers

The kernel GRAPE functions differentiate with respect to raw control coefficients; user-facing parametrisations are handled by thin wrappers that apply exact chain rules:

`grape_xy` — Cartesian X/Y control amplitudes, the standard entry point, called as the cost function of `fmaxnewton`. If a waveform basis $\mathbf{B}$ is supplied (`control.basis`, orthonormal rows), the optimisation variables are the expansion coefficients $\mathbf{w}$, the physical waveform is $\mathbf{u} = \mathbf{w}\mathbf{B}$, and gradients pull back through $\mathbf{B}^{\mathrm{T}}$; Hessians are reordered and transformed with the corresponding tensor contraction. Penalties are computed here and returned in the separated fidelity vector.

`grape_phase` — phase-modulated pulses with a fixed amplitude profile (`control.amplitudes`): variables are the phases $\varphi_k(t)$, the Cartesian waveform is $(A\cos\varphi, A\sin\varphi)$, and the polar chain rule converts gradients and Hessians; only the phase gradient is returned.

`grape_curv` — arbitrary user-defined curvilinear coordinates: the user supplies $u \to x$ maps and their Jacobians, and the wrapper applies them; penalties are evaluated in the rectilinear representation.

`grape_coop` — cooperative pulse pairs (Section 1.11) on top of `grape_phase`.

A separate special-purpose objective, `tgrape()`, differentiates the fidelity with respect to the *interval durations* at a fixed waveform: since $\partial \exp(-i\mathbf{L}\Delta t)/\partial\Delta t = -i\mathbf{L}\exp(-i\mathbf{L}\Delta t)$, the duration gradient needs only generator actions on the stored trajectory. Durations are passed in a user-chosen time unit so that the optimisation variables are of order one — optimisers stall on badly scaled variables.

2.4 Diagnostics and plotting

If `control.plotting` is non-empty, the client calls `ctrl_trajan()` after every objective evaluation with the current waveform, trajectory data, and per-case fidelities. Available panels:

waveform	`xy_controls`, `phi_controls`, `amp_controls`, `frq_controls` (instantaneous frequency)
trajectory	`trajectory`, `correlation_order`, `coherence_order`, `local_each_spin`, `total_each_spin`, `level_populations`
ensemble	`robustness` (fidelity across the ensemble)
time-frequency	`spectrogram`, `time_by_slice`

Trajectory-derived panels force the (large) trajectory stack to be returned from the workers, and all plotting runs on the client every single evaluation — hence the setup-time warning that diagnostic plots are computationally expensive. `control.distplot` optionally applies a distortion chain to the waveform before plotting, so that the display shows what the hardware would emit.

2.5 Waveform distortions

Real pulses are distorted between the waveform generator and the sample — amplifier compression, resonator ringdown, transmission line response. Spinach handles this exactly: `control.distortion` is a cell array of function handles, each mapping a physical waveform to a distorted physical waveform and, on request, returning the Jacobian of that map. During every ensemble case the chain is applied in sequence,

$$\mathbf{u}_{\mathrm{phys}} = D_M(\cdots D_2(D_1(p\,\mathbf{u}))\cdots), \qquad \mathbf{J} = \mathbf{J}_M\cdots\mathbf{J}_2\mathbf{J}_1, \tag{35}$$

and the gradient returned by GRAPE is pulled back through $\mathbf{J}^{\mathrm{T}}$ before the power-level chain rule. Rows of the cell array form an ensemble dimension (distinct distortion realisations to be robust against); columns form the chain stages. Distortion functions may implement anything differentiable — RLC response models, amplifier saturation curves, measured impulse responses — and the supplied Jacobian is verified by the derivative test examples. Because the map composes with the fidelity, only first-order optimisers are permitted when distortions are present (`lbfgs`); analytic Hessians of composed distortion chains are not implemented.

3 Spinach optimal control module architecture

3.1 Data flow

An optimal control calculation passes through the following layers, top to bottom:

layer	role
`optimcon()`	parses and freezes the problem, builds the catalog, publishes worker-resident invariants
`fmaxnewton()`	optimiser iterations and line search
`objeval()`	objective assembly, counters, reshaping
`grape_xy()` and other wrappers	coordinate chain rules, penalties
`ensemble()`	parallel loop over ensemble cases
`grape_liouv()`, `grape_hilb()`	trajectories and propagator derivatives
`step()`, `propagator()`, `dirdiff()`, `aux_mat()`	propagation primitives

The user builds the `control` structure, calls `spin_system=optimcon(spin_system,control)` once, and hands `fmaxnewton` a cost function — almost always `@grape_xy` or `@grape_phase` — together with an initial guess.

3.2 The frozen problem and the worker-resident ensemble

The architectural principle of the module is: *the ensemble is the only parallel dimension, and the evaluation-invariant problem data crosses the process boundary exactly once per optimisation.*

`optimcon()` ends by freezing the problem. It builds the *ensemble case catalog* — the six-column integer array of Section 1.8, one row per case, produced by `ens_catalog()` together with the dimension sizes — and then publishes the complete frozen problem (the entire `spin_system`, control structure included) to the parallel pool as a `parallel.pool.Constant` held in `control.invariants`. The heavy invariant fields — drifts, control operators, offset operators, the trapezium control–control commutator tables, and the Bloch–Siegert response operators, when present — are then removed from the client-side structure, and their names are recorded in `control.frozen_fields`. A `parallel.pool.Constant` constructs without a pool, broadcasts lazily to pools created later, and survives pool deletion and recreation, so serial runs and delayed pool startup work without special cases.

Each objective evaluation calls `ensemble()`, which runs a `parfor` over the catalog rows. Every case fetches the worker-resident frozen problem (a fast local dereference), replaces its control structure wholesale with the live client-side one, re-attaches the fields named in `control.frozen_fields`, assembles its drift (offset terms added per the catalog), applies its phase cycle and distortion chain, calls the serial GRAPE kernel, and returns fidelity, gradient, and Hessian contributions. The client sums the contributions in fixed order and divides by the case count. Per-case propagation is pure serial: the eleven interior `parfor` loops of the previous architecture are gone, along with the time-parallel strategy and all `ValueStore` traffic.

The consequences, measured on a Liouville dimension 21067 problem with 36 ensemble cases and 12 process workers: interprocess traffic fell from 96.97 to 0.20 MB per objective evaluation,

and wall clock from 79.9 to 60.8 s median — the per-evaluation broadcast serialisation was costing about a quarter of the wall time at scale. Only the waveform, the states, and small numerics travel at each evaluation.

Because cases are distributed over workers whole, the parallel efficiency ceiling is the load balance

$$\eta_{\max} = \frac{M}{W\,\lceil M/W \rceil}, \tag{36}$$

for M cases on W workers; `optimcon()` prints this number at setup. It pays to make the case count a multiple of the worker count (e.g. 36 cases on 12 workers gives $\eta_{\max} = 1$; 37 cases give 0.77). Single-case optimisations do not parallelise — for those, the serial kernel is already the optimum at this granularity.

3.3 The mutation contract

Freezing the problem draws a sharp line through the `spin_system.control` structure that every user of sweep drivers should know:

May be overwritten between optimisations (the *live* set): every control field not named in `control.frozen_fields` — in particular `rho_init`, `rho_targ` (same member counts), `pulse_dt` (same length), `pwr_levels`, `offsets` (same value counts), `phase_cycle` (same line count), `distortion` (same rows), `freeze`, `fidelity`, `plotting`, `keyholes`, penalties, weights, bounds, and the prefix and suffix functions. The live structure travels to the workers fresh at every objective evaluation, so overwrites take effect at the next one — this is how parameter sweeps reuse one `optimcon()` setup across many optimisations.

Frozen (changes require a fresh `optimcon()` call): the ensemble *composition* — the member counts of every ensemble dimension, the correlation settings `ens_corrs`, and the budget — and the worker-resident invariants named in `control.frozen_fields`: `drifts`, `operators`, `off_ops`, the trapezium commutator tables `cc_comm` and `cc_comm_idx`, and the Bloch–Siegert `resp_ops`. These fields are physically absent from the client structure after `optimcon()`; touching them, or changing anything that alters the case catalog, raises an immediate error rather than silently running a stale ensemble. The guard literally recomputes the catalog from the current control structure at every evaluation and requires it to be identical to the frozen one, so any catalog-defining drift is caught, including settings added in the future.

Two further architectural guarantees. First, *callbacks see everything*: prefix and suffix functions execute on the workers against the complete frozen problem with the live fields grafted fresh, so a callback that reads `control.parameters` — the designated stash for user data, accepted by `optimcon()` without parsing — or any other control field behaves exactly as it would client-side. Second, *direct low-level calls keep working*: `grape_liouv()` and `grape_hilb()` accept the client-side structure with explicit drift and control arguments, which is how the shipped regression tests drive them.

3.4 Parallel execution model

The module never opens or resizes pools on its own: it uses the pool that exists (`sys.parallel` at `create()` time, or an explicit `parpool` call), and runs serially if there is none — `parfor` with a zero worker cap degrades gracefully. Determinism is worker-count-independent because summation happens client-side in catalog order. The `parallel.pool.Constant` transport is documented by MathWorks for process-local and cluster pools alike; local process pools are the measured configuration.

For very large ensembles, the budget mechanism (Section 1.8) bounds the per-evaluation cost; the random subset is drawn once, at setup, with a fixed seed, so the optimisation surface is

deterministic. For very large state spaces with a single case, parallelism must come from within the propagation primitives (threaded BLAS, GPU arrays supplied as drifts and controls), not from the ensemble loop.

4 Getting started with optimal control in Spinach

4.1 A complete first example

The following script designs a pulse that moves longitudinal magnetisation from the proton to the carbon in a two-spin system through the J-coupling, using controls on both channels — the classic state-to-state transfer. It is complete and runnable as shown.

```
% Parallel pool: eight local process workers
sys.parallel={'processes',8};

% Magnet and spin system
sys.magnet=9.4;
sys.isotopes={'1H','13C'};

% Chemical shifts, ppm, and J-coupling, Hz
inter.zeeman.scalar={1.0,50.0};
inter.coupling.scalar=cell(2);
inter.coupling.scalar{1,2}=140;

% Basis set: full spherical tensor basis, Liouville space
bas.formalism='sphten-liouv';
bas.approximation='none';

% Spinach housekeeping
spin_system=create(sys,inter);
spin_system=basis(spin_system,bas);

% Drift Liouvillian under the NMR rotating-frame assumptions
H=hamiltonian(assume(spin_system,'nmr'));

% Normalised initial and target states
rho_init=state(spin_system,{'Lz'},{1});
rho_init=rho_init/norm(full(rho_init),2);
rho_targ=state(spin_system,{'Lz'},{2});
rho_targ=rho_targ/norm(full(rho_targ),2);

% Control operators: X and Y on both channels
LxH=operator(spin_system,'Lx','1H');
LyH=operator(spin_system,'Ly','1H');
LxC=operator(spin_system,'Lx','13C');
LyC=operator(spin_system,'Ly','13C');

% Control problem specification
control.isotopes={'1H','13C'};                % isotopes under control
control.channels=[1;1;2;2];                   % channel of each operator
control.drifts={{H}};                         % drift ensemble: one member
control.operators={LxH,LyH,LxC,LyC};          % four control channels
control.rho_init={rho_init};                  % one state pair
control.rho_targ={rho_targ};
control.pwr_levels=2*pi*linspace(9e3,11e3,3); % B1 ensemble, rad/s
control.pulse_dt=1e-4*ones(1,50);             % 50 steps of 100 us
control.method='lbfgs';                       % optimiser
control.max_iter=100;                         % iteration cap
```

```
control.plotting={'xy_controls','robustness'}; % live diagnostics

% Freeze the problem
spin_system=optimcon(spin_system,control);

% Random normalised initial guess
guess=randn(4,50)/10;

% Run the optimisation
pulse=fmaxnewton(spin_system,@grape_xy,guess);
```

Everything before the `control` block is ordinary Spinach setup; the optimal control part is the control structure, one `optimcon()` call, and one `fmaxnewton()` call. The result `pulse` is the optimised normalised waveform, one row per control operator, in the same layout as the guess.

4.2 Anatomy of the control structure

Eight fields are mandatory:

field	content
`drifts`	drift generators: a cell array over the ensemble of cell arrays over time — `{{H}}` for one static drift; `{{H1},{H2},...}` for a drift ensemble; each inner array has either one element or one per time point for time-dependent drifts
`operators`	cell array of control operators
`isotopes`	cell array of isotope strings: the isotopes that the control channels address
`channels`	vector with one element per control operator: the index into `isotopes` of the channel that the operator drives
`rho_init`	cell array of initial states (vectors in Liouville space, density matrices in Hilbert space)
`rho_targ`	cell array of target states, same length
`pwr_levels`	row vector of nominal control powers, rad/s — more than one value creates a B_1 robustness ensemble
`pulse_dt`	row vector of interval durations, seconds

Everything else has a sensible default (Section 6). The waveform the optimiser works with is normalised: a value of 1 on a channel means the full nominal power. The guess should therefore be order-of-tenths random numbers or a physically motivated shape scaled the same way; the number of rows equals the number of control operators, the number of columns equals the number of intervals (`numel(pulse_dt)`) for the rectangle integrator, one more for the trapezium integrator.

4.3 Reading the console output

`optimcon()` prints one line per parsed option, the ensemble case count, the pool size, and the parallel efficiency ceiling — worth a glance before a long run. `fmaxnewton()` then prints one line per iteration:

column	meaning
`Iter`	iteration number
`#f, #g, #H, #R`	cumulative objective, gradient, Hessian, and regularisation counts
`fidelity`	current fidelity f
`penalties`	current weighted penalty total
`total`	the maximised objective, $f - \sum w_j P_j$
`alpha`	accepted line search step
`|grad|`	gradient 2-norm
`SDGA`	angle (degrees) between search direction and gradient

A healthy run shows the total climbing, the gradient norm falling, and SDGA well below 90°. The footer reports which termination condition fired. For a state-to-state transfer with the default `real` fidelity, $f = 1$ is a perfect transfer; how close one can get depends on the pulse duration relative to the coupling topology and on the ensemble spread.

4.4 Using the result

The returned waveform is normalised; the physical pulse on channel k is `pwr_levels(m)*pulse(k,:)` for the power level of interest. From here:

- simulate the pulse in any Spinach sequence by adding $\sum_k c_k(t)\,\mathbf{C}_k$ to the drift interval by interval, or by calling the shaped-pulse machinery of the kernel;
- export amplitude and phase per channel: $A = \sqrt{u_X^2 + u_Y^2} \cdot p/(2\pi)$ in Hz and $\varphi = \mathrm{atan2}(u_Y, u_X)$, then resample to the instrument grid;
- verify robustness by evaluating the fidelity over a denser offset/power grid than was optimised over (rebuild the control structure with the dense ensemble, rerun `optimcon()`, and call the cost function once with `max_iter=0` or evaluate `grape_xy` directly);
- continue the optimisation from the result — the returned waveform is a valid guess, including across changes of live settings (Section 3.3).

5 Advanced user manual

5.1 Robustness ensembles

The six ensemble dimensions compose freely; the case count is their product (after correlations and budget). The two workhorse dimensions:

B_1 miscalibration. Several entries in `pwr_levels` optimise the average fidelity over amplifier miscalibration; three to five values spanning ±10% around nominal are typical.

Resonance offset. `control.offsets` is a cell array of row vectors of offset values in Hz, one cell per offset channel, with the matching operators in `control.off_ops` (typically 2π-free $\hat{L}_Z$ operators of the irradiated species — the code multiplies by 2π internally). Multiple channels form a Cartesian product. For a broadband proton pulse: `control.offsets={linspace(-5000,5000,25)}`, `control.off_ops={LzH}`.

For drift ensembles (orientations of a powder, conformer sets, hardware B_0 maps), supply one drift per member; the `rho_drift` and `power_drift` correlations bind the state or power dimension to the drift dimension when members are physically linked (e.g. each crystal orientation carrying its own starting state). When the full product is too expensive, `control.budget` caps the evaluated subset: an integer is a member count, a value ≤ 1 is a fraction; the subset is drawn once with a fixed seed, so the objective stays deterministic. Optimising on a budget subset and *validating on the full grid* is standard practice.

5.2 Time-dependent drifts

Each drift ensemble member may be a cell array with one generator per time point, which is how magic angle spinning, field sweeps, and other programmed drift modulations enter: the propagation cycles through the inner array as it steps through the pulse. All members must have the same number of time slices, and that number must equal the control array length. Rotor-synchronised optimal control under MAS is set up exactly this way.

5.3 Phase cycles

`control.phase_cycle` requires an even number of controls arranged in X/Y pairs. Each row is one cycle line: $[\varphi_{\text{init}}, \varphi_1, \ldots, \varphi_{K/2}, \varphi_{\text{targ}}]$, radians. The optimiser then maximises the average fidelity of the cycle, i.e. designs the pulse so that the *sum* of the phase-shifted experiments performs the transfer while the unwanted pathways cancel. Note that the number of columns is $K/2 + 2$; a receiver phase is folded into φ_{targ}.

5.4 Waveform distortions

Supply `control.distortion` as an $E \times S$ cell array of function handles: S chain stages applied left to right, E ensemble rows. Each handle must implement

```
[wf_out,J]=my_distortion(wf_in)   % J optional, needed for gradients
```

on the *physical* (power-scaled) waveform, returning the Jacobian of the stacked waveform vector on request. Shipped examples cover RLC resonator response chains and the associated Jacobians; the derivative test suite (`examples/fundamentals/derivative_tests`) shows how to verify a hand-written Jacobian against finite differences before trusting an optimisation to it. Distortions restrict the optimiser to `lbfgs`, and combine freely with all other ensemble dimensions — distinct rows model unit-to-unit hardware spread.

5.5 Keyholes, prefix, suffix, and dead time

Keyholes (`control.keyholes`, rectangle integrator only) are projectors applied to the state after every interval: a cell array with one entry per time point, each empty or a linear idempotent function handle (both properties are verified numerically at setup). They implement engineered subspace constraints — decoherence-protected manifolds, coherence-order gating — directly in the trajectory and its derivatives.

Prefix and suffix (`control.prefix` and `control.suffix`) are function handles with the signature

```
rho=f(spin_system,drift,rho)
```

applied to the initial state before the pulse and to the target state before the backward propagation (coded in reverse time), used for fixed preparation and readout blocks around the optimised segment. They execute on the parallel workers against the complete control structure, so they may read any control field, including the free-form `control.parameters` stash.

Dead time (`control.dead_time`) evolves the target backward under the last drift for a fixed delay, producing pulses whose outcome survives a receiver dead time; it is applied with the adjoint generator, so relaxation during the dead time is handled correctly.

5.6 Trapezium integrator in practice

Set `control.integrator='trapezium'` and give the waveform (and guess) $N + 1$ columns for N intervals. The optimised pulse is then natively piecewise-linear — what the instrument plays between DAC points — and the fidelity model error drops from second to fourth order in the

interval length [7], which allows far coarser time grids at equal accuracy. Restrictions: first-order optimisers only (no analytic Hessian), no keyholes, and no steady-state mode. Control–control commutators are precomputed at setup; for commuting control sets (single-channel X/Y pairs commute with themselves only trivially) the extra cost over the rectangle integrator is small.

5.7 Stroboscopic steady-state optimisation

Set `control.steady=true()`. Requirements: `sphten-liouv` formalism, relaxation present, rectangle integrator, first-order optimiser, traceless target, no prefix/suffix/dead time; the supplied initial state is ignored. The optimised functional is the overlap of the target observable with the periodic steady state of the pulse–and–relaxation cycle — the natural objective for steady-state DNP and related experiments; the `steady_orbit` example family shows complete solid effect setups.

5.8 Bloch–Siegert corrections

A resonant control operator is the rotating-frame remnant of a linearly polarised laboratory-frame drive. The counter-rotating component that the rotating-wave approximation discards shifts every energy level in second order of the drive amplitude, and the same drive also shifts every spin that it is not resonant with. Setting `control.bsiegert=true()` keeps these shifts in the optimisation: during problem freezing, `optimcon()` calls `bss_ops()` to build one response operator per control channel,

$$\mathbf{B}_k = \sum_{n \in k} \frac{\mathbf{L}_{\mathrm{Z}}^{(n)}}{2\left(\omega_n + \omega_k\right)} + \sum_{n \notin k} \left(\frac{\gamma_n}{\gamma_k}\right)^2 \frac{\omega_n \mathbf{L}_{\mathrm{Z}}^{(n)}}{\omega_n^2 - \omega_k^2} \tag{37}$$

where $n \in k$ runs over the spins of the isotope that the k-th channel addresses, ω_n are the signed laboratory-frame Zeeman frequencies, and ω_k and γ_k are the carrier frequency and the magnetogyric ratio of the channel isotope. Spins of the channel isotope receive only the never-resonant term, because their resonant term is the control operator itself, which GRAPE propagates exactly; for every other spin the resonant and the never-resonant terms sum into the familiar off-resonant shift $\omega_n\omega_{1n}^2/(\omega_n^2 - \omega_k^2)$, in which $\omega_{1n} = (\gamma_n/\gamma_k)c_{kn}$ is the nutation frequency the drive induces at spin n. The slice generators acquire a quadratic term,

$$\mathbf{L}_n = \mathbf{L}_0 + \sum_k c_{kn}\mathbf{C}_k + \sum_k c_{kn}^2\,\mathbf{B}_k \tag{38}$$

and the derivative direction $\mathbf{C}_k + 2c_{kn}\mathbf{B}_k$ replaces $\mathbf{C}_k$ in the gradient calculation — the gradient stays analytically exact.

The waveform coefficients c_{kn} must therefore be physical nutation frequencies: `optimcon()` refuses control operators on corrected channels that are not unit quadratures $\cos\varphi\,\mathbf{L}_{\mathrm{X}} + \sin\varphi\,\mathbf{L}_{\mathrm{Y}}$ of their channel isotope. Carrier frequencies are taken on resonance with each channel isotope at the current magnet field; zero and degenerate carrier denominators are refused, as are the trapezium integrator and the exact-Hessian optimisers. The response operators are stored without clean-up because their matrix elements scale as inverse Zeeman frequencies.

To replay an optimised pulse outside the optimiser under the same slice generators, `bloch_siegert()` appends the response operators to the control operator list as virtual channels with squared coefficient vectors, ready for the piecewise-constant methods of `shaped_pulse_xy()`:

```
% Amplitudes in rad/s from the normalised waveform
coefs=mat2cell(pwr*pulse,[1 1]);

% Append the Bloch-Siegert virtual channels
[ops,coefs]=bloch_siegert(spin_system,{Lx,Ly},coefs);

% Propagate with a piecewise-constant method
```

```
rho=shaped_pulse_xy(spin_system,H,ops,coefs,...
                    control.pulse_dt,rho,'expv-pwc');
```

The `examples/optimal_control/bloch_siegert` folder demonstrates the corrections on transfer fidelities as a function of drive power, on spectator-spin Ramsey phases, and on the classic 1940 level-shift experiment; `difdiff_bs_rect.m` in the derivative test folder verifies the corrected gradients against finite differences.

5.9 Second-order methods

`newton` and `goodwin` converge in far fewer iterations than LBFGS once inside the quadratic basin, at a much higher per-iteration cost ($O(K^2N)$ auxiliary propagations for the same-interval blocks plus the cross-interval assembly, against $O(KN)$ for the gradient). They pay off when the fidelity landscape is stiff — strongly coupled targets, tight bounds through penalties, ill-conditioned bases — and on modest state-space dimensions where propagator matrices are affordable. The RFO regularisation makes raw saddle-ridden GRAPE Hessians usable; the regularisation iteration count appears in the `#R` column. Goodwin assembly additionally requires all generators Hermitian and is the faster of the two when its cumulative propagators fit in memory. A practical pattern is LBFGS to $f \sim 0.9$, then Newton to convergence — run one optimisation, overwrite `control.method` is *not* possible (method is frozen); instead re-run `optimcon()` with the new method and pass the LBFGS result as the guess.

5.10 Penalties, bounds, and freeze masks

The default is SNS with weight 100 and bounds ± 1 — soft clipping at nominal power. To change the admissible corridor, set `control.u_bound/l_bound` (scalars, or arrays shaped like the waveform for time- and channel-dependent envelopes). For phase-modulated pulses with a fixed amplitude profile, bounds are meaningless (amplitude is fixed); for amplitude-limited phase-agile hardware, use `SNSA`, which caps $\sqrt{u_X^2 + u_Y^2}$ instead of the Cartesian components. `DNS` with a modest weight produces smooth, hardware-friendly waveforms; combining `{'SNS','DNS'}` with weights like `[100 10]` is a common choice. Weights are part of the objective: check the `penalties` column stays small compared to the fidelity, otherwise the optimiser is fighting the penalty rather than the physics.

`control.freeze` is a logical array of the waveform shape: true elements are locked at their guess values — used to pin pulse edges to zero, protect hand-designed segments, or optimise one channel at a time. For `grape_phase`, the freeze mask has the phase-profile shape and is expanded to the Cartesian pairs internally.

5.11 Sweeps and re-optimisation

The mutation contract (Section 3.3) is designed for sweep drivers: after one `optimcon()` call, loops may overwrite live numerical fields — pulse durations, powers, offset values, phase cycles, states, penalties, bounds, plotting — and re-run `fmaxnewton()` each time, typically warm-starting from the previous result. Composition changes (member counts, correlations, budget) and generator changes error out immediately; rebuild the control structure and call `optimcon()` again — it is cheap relative to any optimisation. For unattended long runs, set `control.checkpoint`; for post-mortem analysis of convergence, `control.video_file` records the diagnostic figure per iteration.

5.12 Parallel performance tuning

The rules of thumb that follow from the architecture:

- Aim for a case count that is a multiple of the worker count; the setup printout shows the achievable ceiling.
- More workers than cases is waste; fewer, heavier cases per worker amortise the fixed per-evaluation costs better than many light ones.
- The one-off `optimcon()` Constant broadcast scales with the size of drifts and operators; per-evaluation traffic does not. Sweep drivers that re-run `optimcon()` in a loop pay the broadcast each time — restructure so that composition changes are outside the hot loop where possible.
- Diagnostics cost real time every evaluation: switch `control.plotting` off for production runs and use `robustness` plots only while exploring.
- On a single case, pool workers are idle by design; large single-system problems should invest in threaded BLAS or GPU operators instead.

6 Complete function and option reference

6.1 `optimcon()` options: required fields

field	type	meaning and constraints
`drifts`	cell of cells of matrices	drift generators, outer index over the ensemble, inner over time; inner length 1 or `pulse_ntpts`; all members equal inner length; dimensions must match the control operators; Hermitian where unitarity is required
`operators`	cell of matrices	control operators $\mathbf{C}_k$; cleaned up at absorption
`isotopes`	cell of strings	isotopes addressed by the control channels; each must be present in the spin system
`channels`	vector of positive integers	index into `isotopes` for each control operator, one element per operator
`rho_init`	cell	initial states: column vectors (`sphten-liouv`, `zeeman-liouv`, `zeeman-wavef`) or square matrices (`zeeman-hilb`)
`rho_targ`	cell	target states, same count and kind as `rho_init`
`pwr_levels`	row vector	nominal control powers, rad/s; finite, positive; length > 1 creates the power ensemble
`pulse_dt`	row vector	interval durations, seconds; positive; defines `pulse_nsteps`, `pulse_dur`, and `pulse_ntpts` (equal to `pulse_nsteps`, or plus one for the trapezium integrator)

6.2 `optimcon()` options: optional fields

field	default	meaning and constraints
`fidelity`	`'real'`	fidelity functional: `'real'`, `'imag'`, or `'square'`, Eq. (6)
`integrator`	`'rectangle'`	`'rectangle'` (piecewise-constant) or `'trapezium'` (piecewise-linear)
`method`	`'lbfgs'`	optimiser: `'lbfgs'`, `'rbfgs'`, `'newton'`, `'goodwin'`; the last two refuse the trapezium integrator and distortions; `'goodwin'` requires Hermitian generators
`steady`	`false`	stroboscopic steady-state mode (Section 1.10); many restrictions apply
`prefix`, `suffix`	none	function handles `rho=f(spin_system,drift,rho)` applied before the pulse (initial state) and to the target in reverse time
`dead_time`	`0`	receiver dead time, seconds, non-negative; target evolved backwards under the last drift
`offsets` + `off_ops`	none	offset ensembles: cell of row vectors (Hz) and matching cell of offset operators; both together; same channel count; operator dimensions must match controls
`bsiegert`	`false`	logical scalar; enables Bloch–Siegert corrections (Section 5.8): quadratic response operators added to the slice generators and to the gradient; requires first-order optimisers, the rectangle integrator, and unit-quadrature control operators
`amplitudes`	—	fixed amplitude profile for `grape_phase`; validated there
`freeze`	none	logical waveform-shaped mask of elements excluded from optimisation; may not freeze everything
`max_iter`	`100`	iteration cap, non-negative integer; zero evaluates and plots once
`tol_x`	`1e-3`	termination on step 1-norm
`tol_g`	`1e-6`	termination on gradient 2-norm
`n_grads`	`50`	LBFGS/RBFGS history depth, integer ≥ 2
`phase_cycle`	none	cycle lines $[\varphi_{\text{init}}, \varphi_{1..K/2}, \varphi_{\text{targ}}]$, radians; requires an even control count; $K/2+2$ columns
`basis`	none	orthonormal waveform basis, rows are basis functions, `pulse_ntpts` columns; optimisation runs in coefficient space
`keyholes`	none	cell of `pulse_ntpts` entries, each empty or a linear idempotent function handle; rectangle integrator only
`penalties` + `p_weights`	{`'SNS'`}, `100`	penalty types from `'none'`, `'NS'`, `'SNS'`, `'SNSA'`, `'DNS'` with non-negative weights, both together, equal counts

field	default	meaning and constraints
`u_bound`, `l_bound`	`+1`, `-1`	SNS corridor in fractions of the power level; scalars or waveform-shaped arrays; arrays are incompatible with SNSA and with amplitude-profile optimisations
`ens_corrs`	{}	ensemble correlations from `'rho_ens'`, `'rho_drift'`, `'power_drift'`
`budget`	`Inf`	ensemble budget: integer member count (> 1) or fraction (≤ 1); reproducible random subset
`plotting`	{}	diagnostic panels (Section 2.4); expensive
`distplot`	{}	distortion chain applied to the plotted waveform only
`distortion`	none	$E \times S$ cell of distortion function handles with Jacobians; forces `lbfgs`
`traj_opts`	{}	{`'average'`} returns the ensemble-averaged trajectory
`checkpoint`	none	checkpoint file name in the scratch directory; saves after every accepted step, restarts if present
`video_file`	none	MJPEG video of the diagnostic figure; requires plotting
`parameters`	none	free-form user data, absorbed without parsing; visible to callbacks everywhere

Unrecognised fields are reported by name and refused — a typo cannot silently become a no-op.

6.3 Fields created by `optimcon()`

For sweep drivers and callback authors, the frozen structure contains, besides the absorbed options: `ncontrols` and `ndrifts` (counts), `pulse_dur`, `pulse_nsteps`, `pulse_ntpts` (timing), `catalog` and `ens_sizes` (the ensemble case catalog and dimension sizes), `invariants` (the worker-resident frozen problem, a `parallel.pool.Constant`), `frozen_fields` (the names of the worker-resident invariant fields), the line search and regularisation constants (`ls_c1`, `ls_c2`, `ls_tau1`, `ls_tau2`, `ls_tau3`, `reg_max_iter`, `reg_alpha`, `reg_phi`, `reg_max_cond`), and the parsed forms of every option above. The fields named in `frozen_fields` — `drifts`, `operators`, `off_ops`, and, when present, the trapezium commutator tables `cc_comm` and `cc_comm_idx` and the Bloch–Siegert response operators `resp_ops` — are *absent* from the returned structure: they live on the workers.

6.4 Public functions

`spin_system=optimcon(spin_system,control)` — parses, validates, and freezes the optimal control problem; builds the ensemble catalog; publishes the worker-resident invariants.

`[x,data]=fmaxnewton(spin_system,cost_function,guess)` — maximises the objective from `guess`; returns the optimal waveform and a diagnostics structure (`data.count.*` counters, `data.fx_sep_pen` fidelity-and-penalties vector of the last evaluation, `data.traj_data` trajectory data).

`[traj,fid,grad,hess]=grape_xy(waveform,spin_system)` — Cartesian cost function: ensemble fidelity with penalties separated (`fid(1)` fidelity, `fid(2:end)` weighted penalties; gradients and Hessians stacked accordingly in the third dimension).

`[traj,fid,grad,hess]=grape_phase(phi,spin_system)` — phase-modulated cost function over a fixed amplitude profile.

`[traj,fid,df_du]=grape_curv(u,u2x,dx_du,spin_system)` — curvilinear-coordinate cost function with user Jacobians.

`[traj,fid,grad]=grape_coop(phi,spin_system)` — cooperative pulse pairs; the two phase profiles are stacked vertically.

`[traj,fid,grad,hess]=ensemble(waveform,spin_system)` — the parallel ensemble average of the bare fidelity (no penalties); called by the wrappers, callable directly.

`tgrape()` — `[fid,grad]=tgrape(spin_system,drift,controls, waveform,dt_grid,time_unit,rho_init,r` fidelity and its gradient with respect to interval durations at a fixed waveform.

`[ops,coefs]=bloch_siegert(spin_system,ops,coefs)` — appends the Bloch–Siegert response operator of each control channel as a virtual channel whose coefficient vector is the square of that channel's amplitude vector; the outputs feed the piecewise-constant methods of `shaped_pulse_xy()`.

`[catalog,ens_sizes]=ens_catalog(control)` — the ensemble case catalog: one row per case, columns indexing the state pair, drift, power, offset combination, phase cycle line, and distortion row.

`[p,pg,ph]=penalty(wf,type,fb,cb)` — penalty value, gradient, and Hessian for a waveform between floor and ceiling bounds.

`ctrl_trajan(spin_system,waveform,traj,fids)` — diagnostic plotting; called internally, configured through `control.plotting`.

6.5 Low-level and internal functions

`grape_liouv()` and `grape_hilb()` are the serial GRAPE kernels (trajectories, gradients, Hessians for one ensemble case); they accept the client-side structure with explicit drift and control arguments and are the right entry point for custom ensemble schemes and for regression tests. `aux_mat()` builds the trapezium auxiliary matrices, Eqs. (24)–(25); `dirdiff()` (kernel utilities) implements the N-block directional derivatives; `objeval()` assembles the total objective; `lbfgs()`, `bfgs()`, `hessreg()`, `bracketing()`, `sectioning()`, and `cubic_interp()` are the optimiser internals; `fapt2sfo()` and `inst_freq()` support the diagnostics; `bss_ops()` builds the Bloch–Siegert response operators from the channel isotope list and the carrier frequencies.

6.6 Compatibility matrix

feature	`rectangle`	`trapezium`
LBFGS / RBFGS	yes	yes
Newton / Goodwin Hessians	yes	no
keyholes	yes	no
waveform distortions	LBFGS only	LBFGS only
Bloch–Siegert corrections	LBFGS/RBFGS only	no
stroboscopic steady state	yes (see below)	no
phase cycles, offsets, powers, drifts ensembles	yes	yes
prefix / suffix / dead time	yes	yes (not steady)
waveform basis	yes	yes

Steady-state mode additionally requires `sphten-liouv`, a first-order optimiser, a traceless target, and no prefix/suffix/dead-time; Hilbert space (`zeeman-hilb`), wavefunction formalism, and the Goodwin method require all generators Hermitian; phase cycles require an even number of controls; SNSA and amplitude-profile optimisations require scalar bounds; array bounds require SNS.

References

[1] N. Khaneja, T. Reiss, C. Kehlet, T. Schulte-Herbrüggen, S.J. Glaser, *Optimal control of coupled spin dynamics: design of NMR pulse sequences by gradient ascent algorithms*, J. Magn. Reson. **172** (2005) 296–305.

[2] P. de Fouquieres, S.G. Schirmer, S.J. Glaser, I. Kuprov, *Second order gradient ascent pulse engineering*, J. Magn. Reson. **212** (2011) 412–417.

[3] D.L. Goodwin, I. Kuprov, *Auxiliary matrix formalism for interaction representation transformations, optimal control, and spin relaxation theories*, J. Chem. Phys. **143** (2015) 084113.

[4] D.L. Goodwin, I. Kuprov, *Modified Newton–Raphson GRAPE methods for optimal control of spin systems*, J. Chem. Phys. **144** (2016) 204107.

[5] I. Najfeld, T.F. Havel, *Derivatives of the matrix exponential and their computation*, Adv. Appl. Math. **16** (1995) 321–375.

[6] C.F. Van Loan, *Computing integrals involving the matrix exponential*, IEEE Trans. Autom. Control **23** (1978) 395–404.

[7] U. Rasulov, A. Acharya, M. Carravetta, G. Mathies, I. Kuprov, *Simulation and design of shaped pulses beyond the piecewise-constant approximation*, J. Magn. Reson. **353** (2023) 107478.

[8] A. Iserles, S.P. Nørsett, *On the solution of linear differential equations in Lie groups*, Phil. Trans. R. Soc. Lond. A **357** (1999) 983–1019.

[9] D.C. Liu, J. Nocedal, *On the limited memory BFGS method for large scale optimization*, Math. Program. **45** (1989) 503–528.

[10] A. Banerjee, N. Adams, J. Simons, R. Shepard, *Search for stationary points on surfaces*, J. Phys. Chem. **89** (1985) 52–57.

[11] R. Fletcher, *Practical Methods of Optimization*, 2nd ed., Wiley, 2000.

[12] H.J. Hogben, M. Krzystyniak, G.T.P. Charnock, P.J. Hore, I. Kuprov, *Spinach — a software library for simulation of spin dynamics in large spin systems*, J. Magn. Reson. **208** (2011) 179–194.

[13] I. Kuprov, *Spin: From Basic Symmetries to Quantum Optimal Control*, Springer, 2023.